B12702066 王家穎、B12701101 張舜喆、B12703014 陳韋諺、B12703064 鄭安希、B11702078 周若行、B12703046 蘇苡綸

壹、主題

Magic & Knights: Medieval Combats 以格鬥遊戲為主題，由兩位玩家進行格鬥比賽。每個玩家可以選擇一個角色，進行一對一的戰鬥，直到其中一方被擊敗。此款遊戲強調動作與策略，玩家須透過精確的控制和戰術來擊敗對手。

貳、創作理念

格鬥遊戲為多數組員的喜好之一，不同的角色的屬性與技能設計、操作的難易度等，皆為遊戲增添了挑戰性。遊戲的設計注重流暢和逼真的動作，除了每個角色的動作由不同的動畫呈現，更搭配音效來增強打擊感和技能效果，使戰鬥過程更加生動，玩家有更好的沉浸感。同時，我們的遊戲創作注重操作手感與平衡性的調整，按鍵操作設計順暢，並根據每個角色的屬性調整攻擊力、生命值等，避免出現過度失衡，提供更有挑戰性的對戰體驗。

參、系統設計

遊戲分成選角和對打兩個部分，開始畫面首先提供操作說明以及角色能力，玩家依喜好選出玩家 1 與玩家 2，接著進入對打部分。對打結束後記錄勝負，並於結束畫面詢問玩家是否欲進行下一場對打。

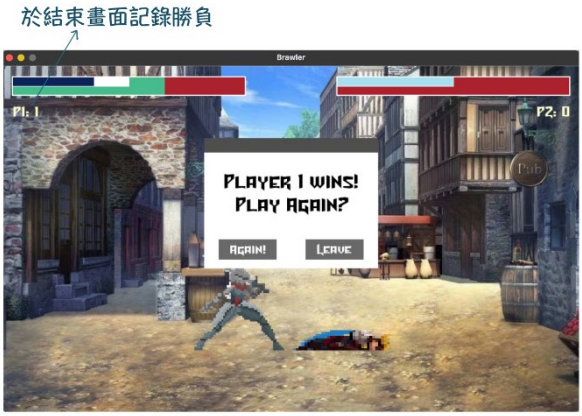
一、 開始畫面

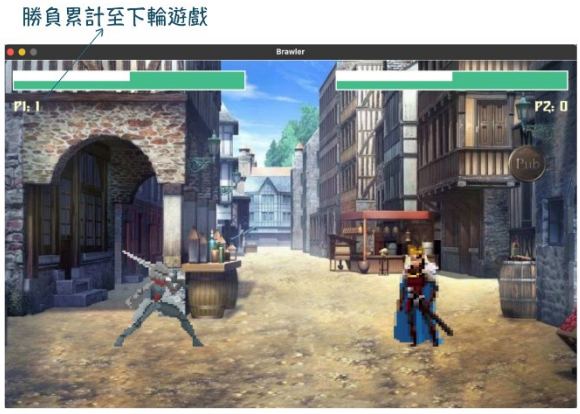
二、 選角

三、 對打

玩家可做前、後、跳三種移動操作，發動兩種攻擊方式。一是沒有發動條件限制的普攻，並同時累積魔力，被攻擊的玩家會累積較多魔力；二是需要魔力量累積的大絕，擁有較大的攻擊力。

四、 結束畫面





肆、程式說明

一、主程式碼

甲、基本遊戲設定

- 設置遊戲視窗和標題
- 定義顏色常量
- 匯入角色、背景圖像
- 設定字體和遊戲狀態
- 定義繪製文字和背景的函數
- 定義角色位置和大小

乙、匯入角色動畫圖

```python
for idx, path in enumerate(animation_paths):
    animation_frames = []
    sprite_sheet = pygame.image.load(path).convert_alpha()
    frame_count = frame_counts[idx]
    frame_height = sprite_sheet.get_height()
    frame_width = sprite_sheet.get_width() // frame_count

    for i in range(frame_count):
        frame = sprite_sheet.subsurface((i * frame_width, 0, frame_width, frame_height))
        animation_frames.append(frame)
    character_animations.append(animation_frames)
```

- 以條狀圖片匯入各角色每個動作的逐幀圖片
- 匯入 sprite_sheet 並轉換為具有透明背景的圖像匯入 sprite_sheet 並轉換為具有透明背景的圖像
- 根據索引從 frame_counts 獲取 sprite_sheet 的幀數

丙、選角

```python
#Function for character select
def draw_character_select():
    draw_character_select_bg()
    if not player1_selected:
        draw_text("Player 1: Select Your Warrior", font, WHITE, 20, 20)
    elif not player2_selected:
        draw_text("Player 2: Select Your Warrior", font, WHITE, 20, 20)

    # Display character images in a single row centered on the screen
    for i, character_image in enumerate(character_images):
        x = CHARACTER_START_X + i * (CHARACTER_SIZE[0] + CHARACTER_GAP)
        y = CHARACTER_START_Y

        if player1_selected and not player2_selected and i == player1_character - 1:
            continue  # Skip the character already selected by player 1

        # Scale
        character_image_scaled = pygame.transform.scale(character_image, CHARACTER_SIZE)
        screen.blit(character_image_scaled, (x, y))
        draw_text(f"{i + 1}", font, WHITE, x, y + CHARACTER_SIZE[1] + 5)
```

- 在屏幕中心顯示操作說明和角色圖片
  - ✧ 使用 enumerate 函數來遍歷 character_images 列表
    - 每次迭代會返回一個元組（i, character_image）
    - i 是索引；character_image 是當前索引對應的圖片
  - ✧ x 座標：計算每個角色圖片的水平位置，CHARACTER_START_X 是第一個角色圖片的起始位置，加上 i 乘以圖片寬度和圖片間的間隔（CHARACTER_GAP），讓每個圖片都會被正確地排列在屏幕上
  - ✧ y 座標固定，所有圖片的垂直位置都是相同的，由 CHARACTER_START_Y 確定
  - ✧ 使用 pygame.transform.scale 將動畫幀縮放到所需的大小

- 播放被選擇的角色動畫
  - ✧ x 座標：將動畫幀水平居中顯示在屏幕上
  - ✧ y 座標：計算垂直位置，將動畫幀顯示在 CHARACTER_START_Y 上方一點位置，以確保動畫幀完全顯示
  - ✧ 使用 ANIMATION_VERTICAL_OFFSET 對象來調整不同角色的垂直偏移，可以根據不同角色的需求微調顯示位置

- screen.blit:將動畫依座標繪製在螢幕上

丁、Game Loop

```
#Game loop
running = True
pygame.mixer.music.play(0, 0.0, 5000) #background music
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
            pygame.quit()
            sys.exit()

        if state == "start_screen":

            if event.type == pygame.MOUSEBUTTONDOWN:
                mouse_x, mouse_y = pygame.mouse.get_pos()
                start_button_rect = pygame.Rect(SCREEN_WIDTH // 2 - 140, SCREEN_HEIGHT // 2 + 70, 320, 100)  # Increased size
                if start_button_rect.collidepoint(mouse_x, mouse_y):
                    lock_fx.play()
                    state = "character_select"
                    animation_counter = 0  # Initialize animation counter when entering character select
                    current_frame_index = 0  # Initialize frame index when entering character select
        elif state == "character_select":
            if event.type == pygame.MOUSEBUTTONDOWN:
                mouse_x, mouse_y = pygame.mouse.get_pos()

```

```
                    # Check if a character is clicked
                    for i in range(len(character_images)):
                        x = CHARACTER_START_X + i * (CHARACTER_SIZE[0] + CHARACTER_GAP)
                        y = CHARACTER_START_Y
                        if x < mouse_x < x + CHARACTER_SIZE[0] and y < mouse_y < y + CHARACTER_SIZE[1]:
                            click_fx.play()
                            selected_character = i
                            current_animation_frames = character_animations[selected_character]
                            current_frame_index = 0
                            animation_counter = 0
                            break

                if event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_RETURN and selected_character is not None:
                        if not player1_selected:
                            lock_fx.play()
                            player1_selected = True
                            player1_character = selected_character + 1
                            selected_character = None
                            current_animation_frames = None
                        elif not player2_selected:
                            lock_fx.play()
                            player2_selected = True
                            player2_character = selected_character + 1
                            running = False
```

- 如果在開始畫面，if event.type == pygame.MOUSEBUTTONDOWN 檢測是否點擊了開始按鈕，使用 pygame.mouse.get_pos() 獲取滑鼠位置，並檢查是否點擊了開始按鈕，若是，則切換狀態至 character_select
- 如果在角色選擇畫面，檢測滑鼠點擊的座標位置判斷所選擇的角色
- 如果兩個玩家都選好角色，退出迴圈，更新當前播放的角色動畫
- 檢查是否有玩家戰敗
- 回合結束可選擇再玩一次或離開遊戲
- 檢測玩家滑鼠點選的位置，如果選擇再玩一次，重置 Fighter；如果選擇離開，退出遊戲

二、自定義資料類別 Fighter

甲、初始化

- 初始化 Fighter：
  設定角色屬性(玩家編號、選角順序、位置、圖像資料、能力值等)
- load_images 函數：

```
def load_images(self, sprite_sheet, animation_steps):
  #extract images from spritesheet
  animation_list = []
  for y in range(len(animation_steps)):
    temp_img_list = []
    for x in range(animation_steps[y]):
      temp_img = sprite_sheet[y].subsurface(x * self.size, 0, self.size, self.size)
      temp_img_list.append(pygame.transform.scale(temp_img, (self.size * self.image_scale[0], self.size * self.image_scale[1])))
    animation_list.append(temp_img_list)
  return animation_list
```

  - ✧ 透過自定義 load_images 函數按不同幀數以迴圈提取動畫幀
  - ✧ 建立一個空列表 temp_img_list 用於儲存當前角色該動作的動畫幀
  - ✧ 每個動作提取完畢後將 temp_img_list 加入 animation_list
  - ✧ 讀取動畫時，按當前動作順序找到該動作列表並逐格呈現
  - ✧ 呈現動畫時，透過各角色的 offset 與 scale 來調整角色位置與大小
- 初始化角色狀態、生命值、魔力值等

乙、移動

- 改變 x 座標值讓角色左右移動
- 改變 y 座標值讓角色跳躍，並增添 gravity 變數，讓角色和現實一樣因重力而回到地面
- 確保角色保持在遊戲畫面內
- 讓雙方角色永遠面對彼此
  - ✧ 角色圖片均面向右方，初始化將 player2 翻轉
  - ✧ 如果對手角色在自己的右方，將 flip 設為 False，否則將 flip 設為 True，翻轉角色
  - ✧ 若角色往左方移動，翻轉圖像

丙、攻擊

- 設定冷卻值讓角色無法持續攻擊，當冷卻值大於零，會隨時間遞減直到等於零，方可進行下一次攻擊
- def attack(self, target) 設定角色攻擊距離與血量

- 普攻

```
def attack(self, target):
    if self.attack_cooldown == 0:
        # Execute attack
        self.attacking = True

        attacking_rect = pygame.Rect(
            self.rect.centerx - (self.attackrange * self.rect.width * self.flip),
            self.rect.y,
            self.attackrange * self.rect.width,
            self.rect.height
        )

        if attacking_rect.colliderect(target.rect) and not self.hit:
            self.attack_sound.play()
            target.health -= self.attackpower
            target.magic += 15
            self.magic += 5
            target.hit = True
```

  - ✧ attacking_rect.colliderect(target.rect)確認是否有攻擊到對手，再讓對手扣血
  - ✧ 自己和對手的魔力值增加
- 必殺技
  - ✧ 直到魔力條累積到 100 才可使用
  - ✧ 讓對手扣血扣得更多

丁、更新角色狀態與動畫

```
170     animation_cooldown = 50
171     #update image
172     self.image = self.animation_list[self.action][self.frame_index]
173     #check if enough time has passed since the last update
174     if pygame.time.get_ticks() - self.update_time > animation_cooldown:
175       self.frame_index += 1
176       self.update_time = pygame.time.get_ticks()
177     #check if the animation has finished
178     if self.frame_index >= len(self.animation_list[self.action]):
179       #if the player is dead then end the animation
180       if self.alive == False:
181         self.frame_index = len(self.animation_list[self.action]) - 1
182       else:
183         self.frame_index = 0
184         #check if an attack was executed
185         if self.action == 3 or self.action == 4:
186           self.attacking = False
187           self.attack_cooldown = 20
188         #check if damage was taken
189         if self.action == 5:
190           self.hit = False
191           #if the player was in the middle of an attack, then the attack is stopped
192           self.attacking = False
193           self.attack_cooldown = 20
```

- animation_cooldown = 50 利用設定每幀圖像停留的時間控制動畫速度
- 程式碼第 174 行：確認自從上一幀圖像是否經過足夠時間，則可播放下一張動畫
- 程式碼第 178 行：代表一動作已執行完畢

伍、組員心得

王家穎：

雖然從小到大，團體報告的心得都會說學到合作，但這次專案才讓我真正體會到合作的重要性。從查找網路資料到撰寫程式碼，再到最後完善遊戲，每一步都證明了團隊協作的價值。

我們組內的分工比起平均分配，更像是各自展露長處：有些人負責動畫，有些人負責文件，有些人負責程式碼，有些人負責影片。雖然每個人都有各自的負責項目，但遇到瓶頸時，大家還是會一起討論、集思廣益，共同解決問題。感謝組員們讓我有這麼好的團隊合作經驗，也感謝商管程讓我遇到這麼優秀的組員，學到這麼多的技能！愛大家！

鄭安希：

對打遊戲所需要運用到的程式和作業內容大大不同，光是 pygame 就需要先了解其中的函數，因此從分工開始就面臨許多的難題，不知道如何分工才能讓每個組員都能順利完成自己的部分，合併程式碼時也會因把程式碼分成許多小部分而有些混亂，在 debug 階段也出現我們從未看過的錯誤訊息，然而我也從過程中學習到很多，更是體悟到老師所說查資料的重要性，程式的世界無遠弗屆，而每個人也都只會選擇某個方面繼續專精，所以遇到自己沒寫過的程式碼是再正常不過的事，這次我從零開始了解如何用 pygame 寫程式，透過了解類似的遊戲、查詢每個函數的用法，達到我們想要的目標，我也學會如何讓自己的程式碼讓別人更容易懂，寫作業時只要自己看得懂自己的程式碼，能夠 debug，且最後會跑出正確的答案即可，但在分工合作時，若是看不懂彼此的程式碼，在合併的階段就會面臨到困難，或是無法找出錯誤並修正，在寫程式時我們也持續發現當初沒有考慮到的問題，只能透過不斷討論與修正讓遊戲更完整，溝通和合作是專案裡不可或缺的一環，對我來說，要單打獨鬥製作出一個完美的遊戲是不可能的，小組中大家負責各自擅長的工作讓遊戲最終能完整的呈現。

張舜喆：

在做專案前因為沒有寫遊戲、實際操作 pygame 的經驗，連概念都很模糊所以感到很焦慮。好在實際操作時，透過查找資料和跟組員間互相學習還是一步一步完成了專案。我也了解到即使沒有很優秀的程式設計能力，只要肯學習、不偷懶，在一行一行了解別人的程式碼間也能有很大的收穫。

蘇苡綸：

此次的專案主題有很多與之前在課程中不一樣的程式語法，也導致我們在分配工作前便需要先了解 pygame 的應用方式、動畫的生成等。本來以為只要合併大家所寫的程式碼，應該不會花過久的時間便能完成，沒想到合併後出現超多問題需要一一解決，好幾位組員為了 debug 都熬夜到兩三點。最後成功生成遊戲後，便又繼續修改讓遊戲看起來更精緻。這次的分組報告是我當學生以來最不一樣的經驗，此項專案是一個人無法完成的，需仰賴大量的團隊合作、分工與討論，謝謝組員讓我有了一次特別的分組經驗。

陳韋諺：

雖然我目前在大學分組合作的次數不多，但每次的經驗都不錯，這次讓我更了解分組合作的價值。小組初期，組員各負責了一小部份程式碼，經過後來的整合討論後，一部份的人轉以文件為主，另一部份則繼續負責程式部分。這樣的分工極大化了我們的效率，在我把 bug 處理好時，文件組的同學已經把文件架構完成了，這樣的速度讓平常都壓線完成作業的我大受震撼，這就是適才適所的重要吧。同時，不同同學在影片製作、文件編輯、或程式方面的才能，讓我看到身旁的同學都臥虎藏龍，這樣的合作兼學習機會十分寶貴，不可多得。

周若行：

這次的小組作業是少數幾次非常順利的經歷之一！原本以為程式作業的分工會很複雜且不容易，但每位組員都完美地認領並消化了自己擅長的部分，並在期限內完成了自己的工作。在實體會議中，我們也高效地整合了所有人的程式碼。當有組員提出如何優化遊戲時，總有人會主動表示「這部分我可以負責！」；且在進行遊戲優化的同時，另一部分組員也高效地開始製作影片和文件。謝謝組員們一起讓這次合作順暢無阻！

陸、分工

張舜喆	人物移動；說明文件
周若行	遊戲開始頁面；選角頁面；遊戲操作介紹
王家穎	血量與魔力條；說明影片
陳韋諺	人物必殺技；遊戲平衡性；音效
蘇苡綸	人物普攻；說明文件
鄭安希	人物承受攻擊；說明文件